

MVP 실험 결과 기반 IR Deck 작성

Sheet 1. 한 문장 요약하기

Sheet 2. IR Deck 초안 만들어보기

[부록] IR Deck 목차 및 흐름

[시트 1. 한 문장 요약하기]

No.	구분	입력 가이드	작성 Sheet
1	Hook	- 누가, 어떤 상황에서, 왜 불편을 느끼는가? - 지금 이 말 한 줄로 청중 시선을 잡을 수 있는가?	웹서비스의 마지막 병목은 무엇인가? DB의 업데이트
2	Problem	- 고객의 'Pain Point'를 숫자나 사례로 표현할 수 있는가? - 기존 대안은 왜 해결 못 했나?	1. PostgreSQL은 기본 설정 기준 메모리 비례, 최대 5,000 connection으로 제한되며 Updata 속도 또한 0.151ms ~ 5.245ms으로 가변적이다. 게다가 고경합상태에서는 Retry 76.67%로 비효율적이다. 2. 기존 기술은 DB를 더 크게 만들거나(Replica), 나누거나(Shard), 앞에 무언가(Cache)를 추가하는 방식으로 폭주를 견뎌왔다.
3	Journey	- 어떤 MVP를, 어떤 채널로, 누구에게 테스트했나? - 얼마나 많은 고객이 어떤 행동을 취했나?	1. 프로야구 티켓 운영사에게 Ticket Engine 기반 좌석 예매 MVP를 직접 만나 설명하고 데모시연을 보여주었으며 실제 테스트진행과 협업에 대한 확답을 받았다. 2. 공연기획사 2군데에게 TicketBG의 상용티켓팅 서비스를 직접 만나 설명하고 제품을 보여주었더니 실제 도입에 대한 확답을 받아 9월중에 실시할 예정이다.
4	Solution	- 배운 인사이트(7 - 9강 리플렉션)를 솔루션에 어떻게 녹였나? - 핵심 가치는 무엇인가?	1. PoC 케이스별 수수료정책을 달리 책정하고 해당 고객군의 의견을 반영하여 제안을 하였더니 만족하며 서비스 도입을 결정하였다. 2. DB / Redis의 인프라 비용을 절감하고 절감되는 비용 범위내에서 서비스 수수료를 책정한다. 즉 고객의 이익 개선이 우선이다.
5	Evidence	- 핵심 성장 지표(click, conversion, payment 등)를 하나 고르자. - 어떤 세그먼트에서 가장 유의미했나?	핵심성장지표는 Test Conversion이다. 고객에게 MVP를 제안한 뒤, 실제 테스트·PoC·도입 협의라는 다음 행동으로 전환된 비율로써 4건의 초기 고객 검증을 통해 4건 모두 후속 행동을 끌어냈다. 현재 우리의 주된 세그먼트는 티켓팅이다. 따라서 우리는 티켓이라는 가장 극단적인 고경합 시장에서 문제를 검증했고, 동일한 문제구조를 가진 다른 산업군시장(리워드, 한정판, 예약, 재고, 전자결제 등)으로 확장한다.
6	Next Step	- 이 수치를 넘어섰을 때, 구체적으로 무엇을 할 것인가? - 투자·협업 요청 포인트는?	1. 유료 PoC 3개, 상용고객 1개를 넘어서면 Scale-up을 시작합니다. Engine + Gateway + SDK/API + Admin + Monitoring 을 갖춘 범용엔진의 표준 B2B 제품을 만들 예정이다. 2. 대규모 동시접속 문제를 실제로 가지고 있는 PoC 고객(티켓사 / 프로스포츠 / 공연 / 커머스 / 예약 / 한정판 매) 연결과 상용화·시장진입을 위한 Seed 투자입니다.

[시트 2. IR Deck 초안 만들어보기]

No.	구분	핵심 질문	작성 Sheet
1	오프닝 (Opening)	이 문제는 왜, 누구에게 얼마나 중요한가?	웹서비스의 마지막 병목은 무엇일까요? 바로 DB의 UPDATE입니다. 웹서비스는 수평 확장을 통해 WAS를 늘리고, CDN을 사용하고, 캐시를 추가하며 대부분의 병목을 해결해 왔습니 다. 하지만 수십만 명이 동시에 같은 좌석, 같은 재고, 같은 쿠폰을 변경하려는 순간에는 결국 하나의 상태를 누가 먼저 가져갈 것인지 결정해야 합니다. 우리는 이 마지막 상태결정 병목을 DB 밖으로 꺼냈습니다.
2	문제점 (Problem)	고객은 어떤 고통(Pain Point)을 겪고 있는가?	대규모 요청이 동일한 상태를 동시에 UPDATE할 때 발생하는 고경합(Contention)이 문제다. 기존에는 이를 해결하기 위해 Shard, Replica, Redis로 확장하며 Lock, Transaction, Connection pool를 통 해 동시성을 제어했으며 이는 인프라비용을 야기한다. 그럼에도 불구하고 수십만 단위의 요청을 빠르게 처리하지 못하고, 결국 대기열서비스를 도입하고 있다.
3	해결책 (Solution)	우리는 어떤 솔루션(MVP)으로 실험했고, 무엇을 보여주었는가?	우리는 DB가 수행하던 실시간 상태결정을 별도의 초경량 엔진(버스트가디언)으로 분리했다. 기존에는 사용자 → WAS → DB UPDATE → 상태결정 순이었다면 Burst Guardian은 사용자 → Gateway → State Engine → 결정값 → DB 로 처리한다. 즉, State Engine이 결정하고, DB는 결과만을 기록하는 것으로 바 꾼 결과 Engine 내부 처리 11ns, Gateway 2.16M TPS, 대규모 Integrity Test 통과라는 기술적 수치를 만들었 다. 이를 통해 단일엔진에서 동시처리 수십만~일백만이 가능하며 샤딩시 수천만도 가능하다.
4	시장기회 (Market)	초기 반응이 어떤 시장 기회를 증명했는가?	초기 검증에서 가장 강하게 반응한 시장은 스포츠·공연 티켓팅이다. 이 시장의 특징은 단순하다. 평상시 낮은 트래 픽 → 오픈 순간 폭증 → 한정된 좌석 → 동일 상태에 대규모 경합 즉 Burst Guardian이 해결하려는 문제가 가장 극단적으로 발생한다. 하지만 동일한 문제는 티켓에만 존재하지 않는다. 티켓 → 재고 → 리워드 → 예약 → 전자결제 → 게임으로 확장이 가능하다. 따라서 티켓팅은 최종 시장이 아니라 Beachhead Market이다.

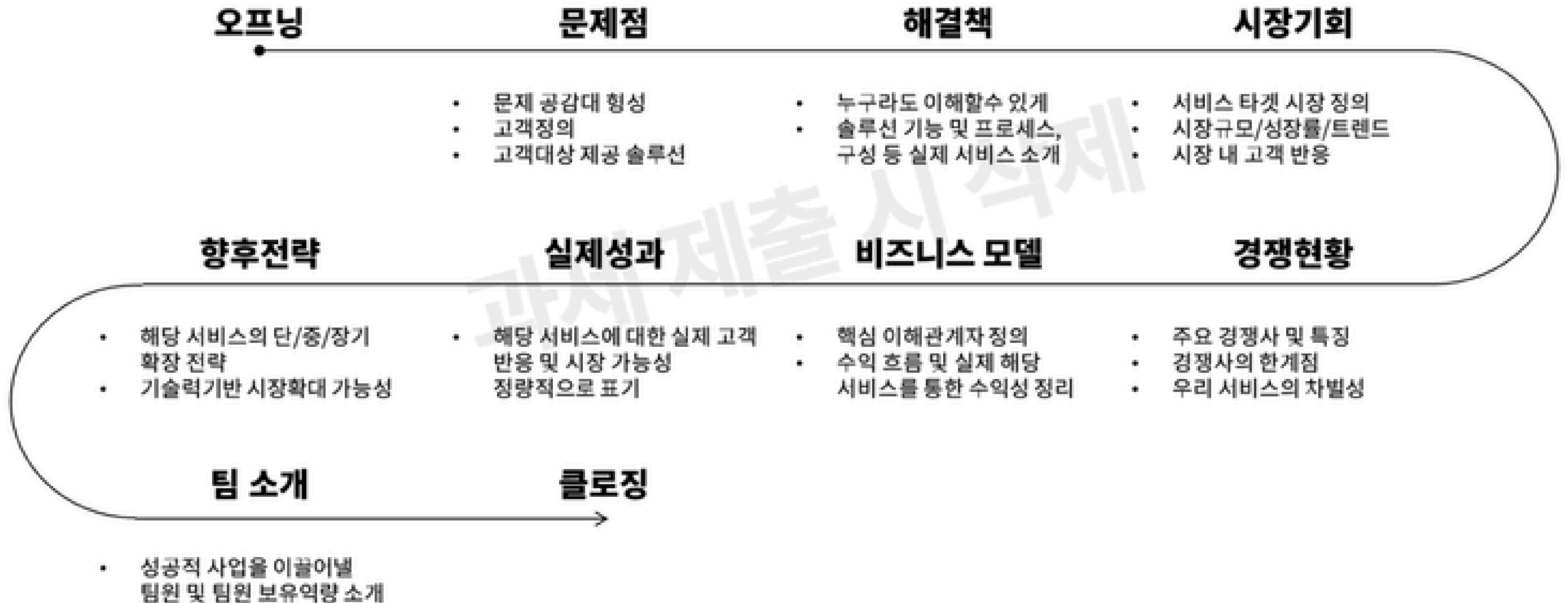
[시트 2. IR Deck 초안 만들어보기]

No.	구분	핵심 질문	작성 Sheet
5	경쟁현황 (Competition)	경쟁사는 누구이며, 우리는 무엇이 다른가?	포지션은 다르지만, 우리의 경쟁이 될만한 것은 DB, Redis, Queue 정도이다. 하지만 우리는 이들과 경쟁을 하려는 것이 아니다. DB, Redis, Queue의 부족한 부분인 실시간 상태변화, 고경합에 대한 특화, 경량화를 통해 보완하는 새로운 레이어가 되려고 한다. 즉, DB, Redis, Queue를 없애는 것이 아니라 줄이게 도와주고 줄여진 비용 범위내에 버스트가디언을 이용하게 함으로써 고객의 인프라 비용을 낮추는 것이다.
6	비즈니스 모델 (Biz Model)	어떻게 돈을 벌며, 수익성은 어디에서 나왔는가?	1. 티켓팅과 리워드분야에서는 확정된 거래에 대하여 1% 또는 약정된 금액의 과금 2. VM / Engine Instance 기반 과금 3. Enterprise에게는 SLA / Private 형 과금
7	실제 성과 (Results)	가장 의미 있는 실험 성과는 무엇인가?	기술적 성과로는 11ns의 Engine 내부 처리, 2.16M TPS의 Gateway 성능, Integrity PASS로 대규모 상태변경 정합성 검증을 한것이며 영업적 성과로는 4건의 초기 고객 검증을 통해 프로야구 관련 2건 테스트 단계 진입 및 협업, 공연기획사 2건 서비스 이용 요청, 리워드 업체 1건 서비스 이용 요청이 있다.
8	향후 전략 (Next Step)	데이터를 바탕으로 다음에는 무엇을 할 것인가?	제품 측면에서는 Ticket Engine을 고객별 SI로 만들지 않고 Engine + Gateway + API/SDK + Monitoring 의 형태로 범용 표준 제품으로 만든다. 그리고 시장측면에서는 스포츠·공연 레퍼런스 확보 후 재고 → 리워드 → 예약 → 전자결제 → 게임 순으로 적용산업을 확장하려 한다. 그리고 상용화·시장진입을 위한 Seed 투자, 고경합 문제를 실제로 보유한 Enterprise PoC 고객 연결을 위해 AC / VC에 제안할 예정이다.

[시트 2. IR Deck 초안 만들어보기]

No.	구분	핵심 질문	작성 Sheet
9	팀 소개 (Team)	왜 우리 팀이 이 사업을 해낼 수 있는가?	팀은 사업과 영업을 맡는 대표, 엔진과 Gateway를 직접 설계한 CTO, KT 현장 CS 경험을 가진 CMO, 엔터테인먼트 전문 변호사인 CLO로 구성돼 있어 개발, 영업, 계약, 현장 운영을 한 팀 안에서 연결할 수 있습니다. 즉 문제를 발견하고 → 직접 만들고 → 부하검증하고 → 고객에게 보여주고 → 다시 제품을 수정하는 전 과정을 내부에서 반복할 수 있다는 것이 저희 팀의 장점입니다.
10	클로징 (Closing)	지금 왜 '투자해야 하는가'?	처음에는 티켓팅 문제라고 생각했습니다. 하지만 고객을 만나고 엔진을 만들면서 발견했습니다. 티켓뿐 아니라, 재고도, 쿠폰도, 예약도, 리워드도 결국 동일한 문제를 가지고 있었습니다. 수많은 사람이 동시에 하나의 상태를 변경하려 합니다. 그래서 웹서비스를 운영하는 기업들은 모두 같은 고민과 인프라비용을 떠안고 있습니다. 그래서 우리는 Ticket Engine에서 출발해 고경합 상태결정을 위한 State Engine을 개발했습니다. 그리고 현재 기술 구현, 성능 검증, 고객 초기반응, 테스트 협의를 마쳤습니다. 이제 우리는 유료 PoC를 거쳐 상용단계로 가고 있습니다.

[부록 : IR Deck 목차 및 흐름]





Orange
Planet



Orange
Park

Orange Planet Foundation. All rights reserved.

